

Nama : Carlos Fadellilah Rama

Kelas : TI-1H

Matkul : Prak.ASD

Minggu : 12

12.2.3 Pertanyaan Percobaan 1 (Operasi Penambahan)

1. Jelaskan perbedaan struktur dan mekanisme traversal antara Single Linked List dan Double Linked List!

- **Struktur:** Single Linked List hanya memiliki satu pointer (next) yang menunjuk ke node berikutnya, sedangkan Double Linked List memiliki dua pointer, yaitu next (ke node selanjutnya) dan prev (ke node sebelumnya).
- **Mekanisme Traversal:** Pada Single Linked List, traversal hanya bisa dilakukan satu arah (maju dari head ke tail). Pada Double Linked List, traversal dapat dilakukan dua arah (maju dari head ke tail, maupun mundur dari tail ke head).

2. Perhatikan class Node, di dalamnya terdapat atribut next dan prev. Jelaskan fungsi masing-masing atribut tersebut pada proses traversal dan manipulasi node!

- **next:** Berfungsi untuk menyimpan alamat memori node berikutnya, memungkinkan program untuk berpindah ke data selanjutnya saat traversal maju.
- **prev:** Berfungsi untuk menyimpan alamat memori node sebelumnya, memungkinkan program untuk kembali ke data sebelumnya tanpa harus mengulang dari head (traversal mundur) serta mempermudah proses penyambungan pointer saat penyisipan atau penghapusan di tengah.

3. Perhatikan konstruktor pada class DoubleLinkedList. Jelaskan fungsi konstruktor tersebut terhadap kondisi awal linked list!

- Konstruktor berfungsi untuk menginisialisasi keadaan awal linked list dengan mengatur atribut head dan tail bernilai null. Ini menandakan bahwa saat objek baru dibuat, linked list dalam keadaan kosong (tidak memiliki node).

4. Mengapa head dan tail harus menunjuk node yang sama ketika linked list masih kosong?

- Ketika node pertama ditambahkan ke list yang kosong, node tersebut bertindak sebagai elemen pertama sekaligus elemen terakhir. Oleh karena itu, head (penanda awal) dan tail (penanda akhir) harus menunjuk ke satu-satunya node yang ada tersebut agar struktur list tetap konsisten.

5. Modifikasi method print() agar menampilkan pesan "Linked List masih kosong" ketika tidak terdapat data!

Java

```
public void print() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong."); [cite: 177]  
        return; [cite: 179]  
    }  
    Node current = head; [cite: 181]  
    while (current != null) { [cite: 182]  
        current.data.tampil(); [cite: 183]  
        current = current.next; [cite: 185]  
    }  
}
```

6. Modifikasi kode program dengan menambahkan method printReverse()!

Java

```
public void printReverse() {  
    if (isEmpty()) {  
        System.out.println("Linked List masih kosong.");  
        return;  
    }  
    Node current = tail; // Mulai dari ekor  
    while (current != null) {  
        current.data.tampil();  
        current = current.prev; // Berpindah ke node sebelumnya  
    }  
}
```

12.3.3 Pertanyaan Percobaan 2 (Operasi Penghapusan)

1. Jelaskan fungsi masing-masing statement berikut pada method removeFirst()!

- **head = head.next;** Memindahkan posisi pointer head ke node kedua, sehingga node pertama yang lama tidak lagi dianggap sebagai awal list.
- **head.prev = null;** Memutus hubungan pointer prev pada node (yang sekarang menjadi) pertama agar menunjuk ke null, secara efektif menghapus referensi ke node lama yang baru saja dikeluarkan.

2. Modifikasi method removeFirst() dan removeLast() agar menampilkan data yang berhasil dihapus!

- **removeFirst() Modifikasi:**

Java

```
public void removeFirst() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
    System.out.println("Data yang dihapus: ");
    head.data.tampil(); // Menampilkan data sebelum dihapus
    if (head == tail) {
        head = tail = null;
    } else {
        head = head.next;
        head.prev = null;
    }
}
```

- **removeLast() Modifikasi:**

Java

```
public void removeLast() {
    if (isEmpty()) {
        System.out.println("Linked List kosong.");
        return;
    }
}
```

```
System.out.println("Data yang dihapus: ");  
tail.data.tampil(); // Menampilkan data sebelum dihapus  
if (head == tail) {  
    head = tail = null;  
} else {  
    tail = tail.prev;  
    tail.next = null;  
}  
}
```